



NATIONAL TECHNICAL UNIVERSITY OF ATHENS
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
DIVISION OF COMPUTER SCIENCE

Continuous Machine Learning for Cooperative, Connected and Automated Mobility applications

DIPLOMA THESIS

of

GEORGIOS A. KYRIAKOPOULOS

Supervisor: Panayiotis Tsanakas
Professor

Athens, October 2025



NATIONAL TECHNICAL UNIVERSITY OF ATHENS
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
DIVISION OF COMPUTER SCIENCE

Continuous Machine Learning for Cooperative, Connected and Automated Mobility applications

DIPLOMA THESIS

of

GEORGIOS A. KYRIAKOPOULOS

Supervisor: Panayiotis Tsanakas
Professor

Approved by the three-member scientific committee on the 30th of October 2025

.....
Panayiotis Tsanakas
Professor NTUA

.....
Andreas-Georgios Stafylopatis
Professor Emeritus NTUA

.....
Georgios Alexandridis
Assistant Professor NKUA

Athens, October 2025

.....
Georgios A. Kyriakopoulos
Graduate of School of Electrical and Computer Engineering,
National Technical University of Athens

Copyright © **Georgios A. Kyriakopoulos, 2025.**
All rights reserved.

Copying, storage, and distribution of this work, in whole or in part, for commercial purposes is prohibited. Reproduction, storage, and distribution for non-profit, educational, or research purposes is permitted, provided that the source is cited and this notice is retained. Inquiries regarding the use of this work for commercial purposes should be directed to the author.

The views and conclusions expressed in this thesis represent those of the author and do not necessarily reflect the official positions of the National Technical University of Athens.

Περίληψη

Λέξεις Κλειδιά:

Abstract

Keywords:

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, **Professor Panayiotis Tsanakas**, for giving me the opportunity to conduct my thesis in a field that has been of great personal interest to me.

I would also like to thank **Dr. Georgios Drainakis** and **Dr. Panagiotis Pantazopoulos** of the **I-SENSE Research Group of ICCS/NTUA** for their valuable feedback and guidance towards the completion of this work.

I am grateful to my family and friends for their support and encouragement throughout this important chapter of my life, especially **Valentina** and **Artemis**, as well as **Nick**, **Apostolis**, **Alex**, **Christini**, and **Elina**.

Finally, and most importantly, I would like to thank **Serafeim** and **George**, with whom I shared the vision, the challenges, and the excitement of this work. Their collaboration and commitment were essential in shaping this thesis, and I truly appreciate sharing this journey with them.

Contents

Contents	11
List of Figures	15
List of Tables	17
Extended Abstract in Greek	19
1 Introduction	21
1.1 Problem Statement	21
1.2 Motivation	21
1.3 Thesis Goal	22
1.4 Contributions	22
1.5 Thesis Structure	23
2 Background and Related Work	25
2.1 Background Concepts	25
2.1.1 Machine Learning for Tabular Regression	25
2.1.2 Concept Drift	26
2.1.3 SUMO Traffic Simulation	28
2.1.4 Platform Technologies	28
2.2 Related Work	30
2.2.1 Drift Detection Tools and Frameworks	30
2.2.2 Machine Learning for ETA Prediction	30
2.2.3 Simulation-Based Evaluation	31
2.3 Positioning and Novelty	32
3 Methodology	33
3.1 Overview	33
3.2 Dataset Generation	33
3.2.1 Geographic Scope and Network Construction	33
3.2.2 Traffic Demand Generation	34
3.2.3 Drift Scenario Design	35
3.2.4 Failed Approaches and Lessons Learned	36
3.2.5 Simulation Execution and Data Export	36
3.2.6 Dataset Characteristics	37
3.2.7 Reproducibility and Public Release	38

3.3	ETA Model Development	38
3.3.1	Problem Formulation	38
3.3.2	Feature Engineering	38
3.3.3	Model Selection	40
3.3.4	Hyperparameter Tuning	40
3.3.5	Feature Calibrator for Online Predictions	41
3.4	Platform Architecture	41
3.4.1	Architectural Overview	42
3.4.2	Backend Service	42
3.4.3	Predictor Services	43
3.4.4	Drift Detection Service	44
3.4.5	Frontend Service	45
3.4.6	Technology Stack	46
3.4.7	Deployment and Configuration	46
3.4.8	Extensibility Patterns	47
4	Implementation	49
4.1	Time-Lapse Simulation Workflow	49
4.1.1	Simulation Timeline	49
4.1.2	Data Flow	49
4.2	Retraining Workflow	51
4.2.1	Trigger Condition	51
4.2.2	Job Submission	51
4.2.3	Background Training	51
4.2.4	Model Hot-Swapping	52
4.2.5	Failure Handling	52
4.3	API Specifications	52
4.3.1	Prediction APIs	52
4.3.2	Drift Detection APIs	53
4.3.3	Control APIs	54
4.4	Operational Considerations	54
4.4.1	Performance Characteristics	54
4.4.2	Resource Usage	55
4.4.3	Scalability	55
4.4.4	Monitoring and Logging	55
4.4.5	Data Management	55
4.4.6	Reproducibility	56
5	Evaluation and Results	57
5.1	Experimental Setup	57
5.1.1	Hardware and Software	57
5.1.2	Datasets	57
5.1.3	Evaluation Metrics	57
5.2	RQ1: Drift Induction via Friction-Based Rain	58
5.2.1	Model Performance on Test vs. Rain	58
5.2.2	Error Distribution Analysis	58
5.3	RQ2: Consensus-Based Drift Detection	59
5.3.1	Time-Lapse Simulation Results	59

5.3.2	Detector Behavior	59
5.3.3	Comparison to Single-Detector Approaches	60
5.4	RQ3: Retraining Effectiveness	61
5.4.1	Model Adaptation Results	61
5.4.2	Performance Recovery Analysis	61
5.4.3	Hot-Swap Verification	61
5.5	RQ4: Feature Engineering Robustness	62
5.5.1	Feature Ablation Study	62
5.5.2	Feature Robustness Under Drift	62
5.6	Model Comparison	63
5.6.1	Algorithm Selection Justification	63
5.7	Dashboard and Monitoring	63
5.8	Limitations and Threats to Validity	64
5.8.1	Internal Validity	64
5.8.2	External Validity	64
5.8.3	Construct Validity	64
6	Conclusion and Future Work	65
6.1	Summary of Contributions	65
6.1.1	Research Answers	65
6.1.2	Technical Contributions	66
6.1.3	Practical Impact	66
6.2	Limitations	66
6.2.1	Simulation Fidelity	66
6.2.2	Drift Scenario Coverage	67
6.2.3	Retraining Strategy	67
6.2.4	Evaluation Scope	67
6.3	Future Work	67
6.3.1	Short-Term Extensions	67
6.3.2	Medium-Term Research	68
6.3.3	Long-Term Vision	69
6.4	Closing Remarks	69
	Bibliography	71

List of Figures

3.4.1 Platform microservices architecture	42
5.2.1 Error distribution comparison	58
5.3.1 MAE evolution during time-lapse simulation (illustrative)	60

List of Tables

3.1	Dataset statistics for train, test, and rain simulations	37
3.2	Hyperparameter search space for LightGBM	41
3.3	Platform technology stack	46
4.1	Platform performance metrics	54
4.2	Per-service resource consumption	55
5.1	ETA model performance on test vs. rain datasets	58
5.2	Drift detection timeline	59
5.3	Individual detector firing times	59
5.4	Single-detector vs. consensus performance	60
5.5	Model versions and performance	61
5.6	Pre- and post-adaptation performance	61
5.7	Feature set ablation results (5-fold CV on training set)	62
5.8	Top 10 features by gain (LightGBM split gain)	63
5.9	Model family comparison (5-fold CV on training set)	63

Extended Abstract in Greek

Chapter 1

Introduction

1.1 Problem Statement

Urban mobility environments are highly dynamic and *non-stationary*, meaning that the statistical properties of traffic data change over time. Machine learning models trained on static historical data often assume a fixed data distribution, an assumption that rarely holds in real city conditions [1]. When deployed in a dynamic urban setting, a model’s performance can silently degrade as driving patterns shift due to *concept drift*, the evolution of the underlying data relationships over time [2]. This drift is triggered by many factors: for example, road network changes (e.g., construction) can alter traffic flow patterns significantly [3]. Even routine variations like weather and seasonal demand swings can lead to abrupt or gradual shifts in vehicle speeds and congestion levels. A model that performs well on dry, low-traffic days may become less accurate during heavy rain or rush-hour peaks if those conditions were not part of its training distribution [4]. the challenge is that static ML models struggle to maintain accuracy in the face of non-stationarity, as their predictive power deteriorates over time unless the models continually adapt to the evolving urban mobility data [1]. Recognizing and addressing this concept drift in cooperative and automated mobility scenarios is therefore a critical problem identified by the research community.

1.2 Motivation

Given the inevitability of concept drift in real-world intelligent transportation systems, there is a clear need for a practical, drift-aware platform tailored to the mobility domain. While concept drift detection and adaptation have been studied in research, existing solutions are often fragmented or confined to laboratory settings [5]. Current tools lack integration of critical components, for instance, traffic simulation, continuous monitoring, and adaptive retraining are rarely all incorporated into one seamless loop. This means practitioners may detect drifts, but they often have no automated way to replay scenarios (e.g. via simulation) or update models on the fly in response to those drifts.

A domain-specific platform would fill this gap by monitoring model performance through time (not just offline metrics), automatically detecting distribution shifts, and triggering model adaptation with minimal human intervention. Such an integrated approach is especially vital in *Cooperative, Connected and Automated Mobility (CCAM)* applications where conditions can change rapidly and safety is paramount. One key use case is *Estimated Time of Arrival (ETA)* prediction

for vehicles, a core service in smart mobility that influences traffic management, logistics, and traveler information [6]. However, many current mobility systems do not continuously recalibrate ETA models as new data (e.g., live traffic feeds or weather updates) arrive. This motivates the creation of a continuous learning platform for mobility: one that integrates traffic simulation (to test “what-if” scenarios and generate synthetic data), live performance dashboards (to observe errors creeping up over time), drift detection algorithms (to raise alerts when the model starts faltering), and an automated retraining pipeline (to deploy updated models when needed). By addressing these needs, we aim to ensure that mobility prediction models remain accurate and robust despite the ever-changing urban environment.

1.3 Thesis Goal

The objective of this thesis is to design, implement, and evaluate a microservice-based platform for *continuous machine learning* in urban mobility applications. In essence, the goal is to create a *drift-aware ML platform* that supports end-to-end monitoring and adaptation of predictive models in a city traffic context.

The platform will be built using a modular microservice architecture to ensure scalability and flexibility, where each component (orchestration, model serving, drift detection, etc.) operates as an independent service. Crucially, the platform is intended to be model-agnostic and domain-specific: it can host any machine learning model for mobility prediction tasks, but this thesis will demonstrate and evaluate it primarily through the lens of an Estimated Time of Arrival (ETA) prediction use case. We focus on ETA because of its importance in CCAM scenarios (helping vehicles and travelers coordinate effectively), and because it provides a tangible benchmark to test how well the platform handles concept drift. The platform will ingest real-time or simulated vehicle telemetry and output ETA predictions, all the while monitoring for performance drops. Upon detecting drift, it will automatically trigger model adaptation (retraining or model swapping) to self-correct the predictions.

Although ETA is the main case study, the system is designed to accommodate additional models, for example, external predictive modules for Fuel Consumption [7] or Number of Stops [8] will be integrated to showcase that multiple concurrent mobility models can coexist and benefit from the shared drift-aware infrastructure. By the end of this thesis, we aim to show that such a platform can be realized and that it effectively maintains model performance over time in a realistic urban traffic setting.

1.4 Contributions

This thesis makes several contributions to the state of the art in continuous machine learning for mobility:

- **Synthetic Dataset Pipeline:** We developed a reproducible, modular data pipeline that generates training and evaluation datasets using synthetic traffic data from Simulation of Urban MObility (SUMO) [9]. Focusing on a case study of central Athens, the pipeline takes an open-source city road network and creates realistic traffic scenarios (vehicles, routes, events) to produce rich labeled data. This approach ensures that experiments are repeatable and tunable (e.g. the pipeline can be re-run for any city region or traffic pattern by adjusting parameters), addressing the shortage of standard drift benchmarks in urban mobility.

- **ETA Prediction Model under Drift:** We designed a feature-engineered ETA prediction model using gradient-boosted decision trees (LightGBM) [10]. The model is trained on the above dataset to estimate travel times for vehicles, and we introduce domain-specific features to improve accuracy. More importantly, the model’s performance is evaluated under simulated drift scenarios (e.g., a sudden shift from normal traffic to heavy rain conditions that reduce vehicle speeds). This allowed us to study how concept drift affects ETA prediction accuracy, and to validate that our model (and platform) can detect and respond to these changes.
- **Time-Lapse Drift Simulation:** We conducted a time-lapse simulation experiment to rigorously assess drift detection and adaptation in an end-to-end fashion. In this setup, the SUMO simulator replays an extended sequence where an initial period of stable conditions is followed by a clear drift event (a transition from dry weather to a storm causing slower traffic). The platform’s drift detectors monitor the real-time prediction error and successfully detect the concept drift when the environment changes. We then measure how the system’s automatic adaptation (triggering model retraining with new data from the rainy period) helps recover the model’s performance. This experiment provides a realistic validation of the platform’s ability to observe, detect, and mitigate drift in a controlled yet lifelike scenario.
- **Drift-Aware Platform Implementation:** We present the design and implementation of a platform for continuous learning in mobility, which is a key practical contribution of this work. The platform includes components for simulation replay (feeding historical or synthetic data through the model as if in real time), live dashboarding of model performance metrics (so that users can visualize concept drift as it happens), pluggable drift detection algorithms (with calibration tools to set detection thresholds and minimize false alarms), and an automatic retraining and deployment pipeline (which seamlessly replaces the old model with an updated one when drift is confirmed). The entire system is built with modern software frameworks (containerized microservices, REST APIs for model serving, and event-driven triggers for retraining) to ensure it can be deployed in real operational environments. By open-sourcing the platform and detailing its architecture, we enable other researchers and practitioners to reproduce our approach and extend it to their own cooperative and automated mobility applications.

1.5 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 2: Background** - Introduces the foundational concepts and tools underlying our work. We review machine learning basics relevant to continuous model updating, the functionalities of the SUMO traffic simulator (for creating mobility scenarios), the notion of concept drift (types of drift and common detection methods), and the platform technologies (microservices, Docker, APIs, etc.) used to build our solution.
- **Chapter 3: Related Work** - Surveys existing literature and systems in areas pertinent to our thesis. We cover drift detection tools and frameworks (e.g. streaming ML libraries and MLOps solutions for model monitoring), examine available urban mobility simulation environments used to study drift, and review prior research on ETA prediction models in intelligent transportation. This chapter highlights how our approach compares to and

innovates upon the current state of the art.

- **Chapter 4: Dataset and Modeling** - Details our methodology for creating the dataset and developing the ETA prediction model. We describe the data generation process using SUMO for the Athens case study, including how traffic demand patterns and drift scenarios (like weather changes) are configured. We then present the ETA model development, including feature engineering, hyperparameter tuning, model selection (why LightGBM) and the overall training procedure and the set up of the experiments.
- **Chapter 5: Platform Architecture** - Provides a comprehensive overview of the drift-aware platform’s design. We enumerate the microservices and their roles, explain how the system ingests data (simulation or extensions to real streams), how drift detection algorithms are integrated, and how the platform orchestrates retraining and deployment of models. Implementation details of key components (such as the real-time dashboard) are given to illustrate how the system works in practice.
- **Chapter 6: Results** - Presents the results of our evaluation. We report the baseline performance of the ETA model under static conditions and then analyze its behavior under induced drift (e.g., performance drop during the simulated rain scenario). We demonstrate the effectiveness of drift detection (e.g., detection delay) and quantify the gains from automatic adaptation (improvement in error after retraining). The results validate the platform’s ability to maintain model accuracy over time.
- **Chapter 7: Conclusions** - Summarizes the contributions and findings of the thesis. We reflect on the success of the continuous learning approach for CCAM applications and any limitations observed. Finally, we outline future research directions, such as improvements to platform scaling and monitoring, or integrating the system with real-time data streams.

Each of these chapters builds upon the motivation and goals outlined in this introduction, collectively advancing the vision of continuous machine learning for cooperative, connected, and automated mobility. By the end, we aim to demonstrate a viable pathway for model lifelong learning in urban transportation systems, ensuring they remain reliable as the world around them changes.

Chapter 2

Background and Related Work

This chapter provides the theoretical foundations and technological context necessary for understanding the drift-aware platform presented in this thesis. We first introduce core concepts from machine learning, concept drift, and traffic simulation (Section 2.1). We then survey existing tools and research relevant to drift detection, mobility prediction, and simulation-based evaluation (Section 2.2), concluding with a discussion of how this thesis extends and integrates these areas (Section 2.3).

2.1 Background Concepts

2.1.1 Machine Learning for Tabular Regression

This thesis focuses on supervised regression tasks where the goal is to learn a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ mapping input features $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^d$ to continuous outputs $y \in \mathcal{Y} \subseteq \mathbb{R}$. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, the learning algorithm minimizes an empirical risk function

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(f(\mathbf{x}_i), y_i) + \lambda R(f) \quad (2.1.1)$$

where L is a loss function (e.g., squared error, absolute error), $R(f)$ is a regularization term, and λ controls the regularization strength.

Gradient Boosting Decision Trees

For tabular data with heterogeneous features, gradient boosting decision trees (GBDT) have consistently demonstrated state-of-the-art performance [chen2016xgboost, 10]. The core idea is to build an ensemble of weak learners (decision trees) sequentially, with each tree fitting the residual errors of the previous ensemble:

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu \cdot h_m(\mathbf{x}) \quad (2.1.2)$$

where h_m is the m -th tree and ν is the learning rate. Modern implementations like LightGBM use histogram-based splitting for computational efficiency and support categorical features directly. The final prediction combines all trees:

$$\hat{y} = f_M(\mathbf{x}) = \sum_{m=1}^M \nu \cdot h_m(\mathbf{x}) \quad (2.1.3)$$

Evaluation Metrics

For regression tasks, multiple metrics assess different aspects of model quality:

- **Mean Absolute Error (MAE)**: $\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ measures average absolute deviation in original units. It is robust to outliers and directly interpretable.
- **Root Mean Squared Error (RMSE)**: $\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$ penalizes large errors more heavily than MAE.
- **Mean Absolute Percentage Error (MAPE)**: $\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$ provides scale-independent assessment but is undefined for $y_i = 0$ and sensitive to small denominators.
- **Coefficient of Determination (R^2)**: $R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$ measures proportion of variance explained, with perfect prediction yielding $R^2 = 1$.

For time-critical applications like real-time ETA prediction, training time and inference latency are also crucial metrics. This thesis reports all four accuracy metrics plus computational performance.

Cross-Validation

To obtain reliable estimates of model performance and avoid overfitting, this research uses k -fold cross-validation [kohavi1995study]. The training set is partitioned into k disjoint folds, and the model is trained k times, each time using $k - 1$ folds for training and one fold for validation. The final performance estimate is the mean across folds.

For ETA prediction, we employ *stratified* cross-validation where folds are constructed to preserve the distribution of trip duration (the target variable). This ensures each fold is representative of the full range of trip types, which is critical when the target distribution is skewed.

2.1.2 Concept Drift

Concept drift refers to changes in the statistical properties of the target variable or its relationship to input features over time [1, 2]. Formally, drift occurs when the joint distribution $P(\mathbf{X}, Y)$ evolves:

$$P_t(\mathbf{X}, Y) \neq P_{t'}(\mathbf{X}, Y) \quad \text{for } t \neq t' \quad (2.1.4)$$

Types of Drift

Drift can manifest in several forms [1]:

- **Real Concept Drift**: The conditional distribution $P(Y|\mathbf{X})$ changes, meaning the same inputs produce different outputs over time. Example: Trip from location A to B takes longer during rush hour.

- **Virtual Drift:** The input distribution $P(\mathbf{X})$ changes but $P(Y|\mathbf{X})$ remains constant. Example: Seasonal changes in origin-destination patterns.
- **Sudden Drift:** Abrupt change at a specific time point. Example: Road closure due to construction.
- **Gradual Drift:** Slow, continuous change over an extended period. Example: Traffic pattern evolution as city grows.
- **Recurring Drift:** Periodic return to previous concepts. Example: Weekday vs. weekend traffic patterns.

The rain scenario in this thesis induces *sudden real concept drift*: at the start of the rain period, road friction changes abruptly, causing the relationship between trip features and duration to shift.

Drift Detection Methods

Numerous algorithms have been proposed for online drift detection [gama2004learning, baena2006early, 4]. Most operate on a stream of performance metrics (e.g., prediction errors) and signal drift when statistical properties of this stream change significantly. Key algorithms include:

- **ADWIN (Adaptive Windowing)** [bifet2007learning]: Maintains an adaptive window of recent errors and detects drift when the mean error in two sub-windows differs significantly. No parameters required except confidence level.
- **Page-Hinkley Test** [page1954continuous]: Cumulative sum control chart that detects increases in the mean error. Requires threshold and acceptable magnitude parameters.
- **KSWIN (Kolmogorov-Smirnov Windowing)** [raab2020reactive]: Applies two-sample Kolmogorov-Smirnov test to recent vs. reference error windows to detect distribution changes.
- **SPC (Statistical Process Control)** [montgomery2009introduction]: Declares drift when consecutive errors exceed control limits based on baseline statistics.

A common challenge is balancing sensitivity (detecting true drift quickly) and specificity (avoiding false alarms on natural variability). This thesis addresses this through *consensus-based detection*: requiring multiple detectors to agree before declaring drift.

Adaptation Strategies

Once drift is detected, several adaptation approaches are possible [1]:

- **Retraining:** Train a new model on recent data, either from scratch or using the old model as initialization.
- **Ensemble Methods:** Maintain multiple models trained on different time windows and combine their predictions adaptively.
- **Incremental Learning:** Update model parameters using new data without full retraining (e.g., online gradient descent).
- **Instance Weighting:** Emphasize recent examples during training to track drift.

This thesis employs targeted retraining: collecting a window of post-drift data and training a new model using reduced ensemble size (proportional to available data) to prevent overfitting. The new model is hot-swapped into production without interrupting prediction serving.

2.1.3 SUMO Traffic Simulation

SUMO (Simulation of Urban MObility) [sumo2012] is an open-source, microscopic traffic simulation platform widely used in research and industry. It models individual vehicle behavior at sub-second resolution, enabling detailed analysis of traffic dynamics.

Key Capabilities

- **Network Import:** Direct import from OpenStreetMap (OSM) with automatic conversion of roads, junctions, and traffic lights.
- **Microscopic Car-Following:** Multiple models (Krauss, IDM, W99) simulate acceleration, braking, and gap maintenance.
- **Route Planning:** Built-in Dijkstra and A* algorithms for static and dynamic routing.
- **Environmental Models:** Fuel consumption (PHEMlight, HBEFA), emissions (CO₂, NO_x), and noise.
- **Friction Device:** Simulates road surface conditions (dry, wet, snow, ice) with speed reduction based on validated models [weber2021friction].
- **Floating Car Data (FCD):** Exports per-vehicle trajectories including position, speed, lane, fuel consumption, and waiting time at each timestep.

Pipeline: OSM to FCD

The typical workflow is:

1. **osmGet:** Download OSM data for a bounding box
2. **netconvert:** Convert OSM to SUMO network with road attributes and traffic lights
3. **randomTrips.py:** Generate origin-destination pairs with temporal distribution
4. **sumo:** Run simulation and export FCD
5. **Post-processing:** Aggregate FCD into trip-level features

This thesis leverages SUMO’s friction device to systematically induce concept drift: by reducing the friction coefficient on all lanes, we simulate degraded road conditions (rain, snow) that affect vehicle speeds and travel times.

2.1.4 Platform Technologies

The drift-aware platform relies on modern software engineering practices to achieve modularity, scalability, and maintainability.

Containerization with Docker

Docker [merkel2014docker] enables packaging applications with their dependencies into lightweight, portable containers. Key benefits:

- **Reproducibility:** Ensures identical execution environments across development, testing, and deployment.
- **Isolation:** Each service runs in its own container with independent dependencies.
- **Orchestration:** Docker Compose [docker-compose] manages multi-container applications with declarative YAML configuration.

The platform uses Docker Compose to orchestrate six services (backend, three predictors, drift detector, summarizer) with defined dependencies and health checks.

FastAPI for REST Services

FastAPI [fastapi] is a modern Python web framework for building APIs with:

- **Automatic Validation:** Pydantic models ensure type safety and data validation at API boundaries.
- **Async Support:** Native async/await for high concurrency.
- **Auto-Generated Documentation:** OpenAPI schemas and interactive documentation.

All backend and predictor services expose RESTful APIs via FastAPI, enabling loose coupling and independent scalability.

Dash for Interactive Dashboards

Plotly Dash [plotly-dash] provides reactive web dashboards in pure Python without JavaScript:

- **Component Library:** Dash Bootstrap Components for modern UI elements.
- **Dash Leaflet:** Interactive maps for geospatial visualization.
- **Callback Architecture:** Declarative reactive programming model.

The frontend service implements two interfaces: an admin dashboard for monitoring drift detection and system state, and a user interface for querying ETA predictions on an interactive map.

Apache Parquet for Data Storage

Apache Parquet [parquet] is a columnar storage format optimized for analytics:

- **Predicate Pushdown:** Efficient filtering without loading full dataset.
- **Compression:** Typically 50-70% smaller than CSV.
- **Schema Evolution:** Add columns without breaking existing code.

Training and test data are stored in Parquet format, enabling fast time-window queries for batch predictions and retraining.

2.2 Related Work

2.2.1 Drift Detection Tools and Frameworks

Several open-source tools address concept drift detection and online learning:

- **River** [river2020]: Python library for online machine learning with a comprehensive collection of drift detectors (ADWIN, Page-Hinkley, KSWIN, HDDM) and incremental learning algorithms. Emphasizes single-pass processing and memory efficiency. This thesis uses River for drift detection but extends it with consensus mechanisms and automatic calibration.
- **scikit-multiflow** [montiel2018scikit]: Predecessor to River, focused on stream learning. Development has largely migrated to River.
- **Alibi Detect** [alibi2020]: Focuses on outlier and distribution shift detection with support for tabular, text, and image data. Provides advanced methods like Maximum Mean Discrepancy (MMD) and learned detectors. Less emphasis on online algorithms suitable for real-time deployment.
- **Evidently AI** [evidently2021]: Monitors ML model performance in production with interactive reports and drift dashboards. Primarily targets batch monitoring rather than online detection.
- **NannyML** [nannyml2022]: Post-deployment model monitoring with emphasis on estimated performance when ground truth is delayed. Good for production monitoring but requires batch processing.

Gap: While these tools provide excellent drift detection primitives, they do not address domain-specific challenges of mobility applications or provide integrated end-to-end platforms combining simulation, detection, and adaptation.

2.2.2 Machine Learning for ETA Prediction

Extensive research addresses travel time prediction using various approaches:

Classical Methods

Early work employed historical averages, linear regression, and ARIMA models [chien2002dynamic, rice2004accurate]. These methods struggle with nonlinear relationships and spatiotemporal dependencies.

Tree-Based Methods

Gradient boosting decision trees have emerged as strong baselines for ETA prediction on tabular data [wang2016travel, woodard2017predicting]:

- Wang et al. [wang2016travel] use XGBoost with spatial clustering and temporal binning features, achieving MAPE < 15% on taxi trajectory data.
- Woodard et al. [woodard2017predicting] demonstrate that LightGBM with engineered features outperforms deep learning models on structured trip data while requiring far less training time.

Deep Learning Approaches

Neural architectures have been explored for capturing complex patterns [derrow2016deep, duan2018approach]:

- Derrow-Pinion et al. [derrow2016deep] use LSTMs with attention mechanisms for sequence-to-sequence ETA prediction on Google Maps data.
- Duan et al. [duan2018approach] propose Graph Neural Networks to model road network structure explicitly.

However, deep learning typically requires massive datasets and substantial computational resources. For moderate-scale problems with well-defined features, gradient boosting remains competitive or superior [shwartz2022tabular].

Feature Engineering Practices

Successful ETA prediction systems rely on domain-specific feature engineering [wang2016travel, woodard2017predicting]:

- **Spatial:** Euclidean distance, route length, detour factor, distance from city center
- **Temporal:** Hour of day, day of week, rush hour indicators, cyclical encoding
- **Clustering:** Spatial zones via K-Means or grid discretization
- **Contextual:** Weather, events, historical averages for time and location

This thesis systematically evaluates feature sets and their contributions using ablation studies and importance rankings.

2.2.3 Simulation-Based Evaluation

Traffic simulation provides controlled environments for evaluating ML systems under various conditions [sumo2012, krajzewicz2006sumo].

SUMO in ML Research

SUMO has been used to generate training data for:

- Traffic signal control using reinforcement learning [chu2020multiagent]
- Vehicle routing optimization [zhang2018route]
- Emission prediction models [wang2008estimating]

However, these applications focus on static datasets rather than dynamic drift scenarios.

Drift Manufacturing

Prior work on simulated concept drift has primarily used synthetic datasets with artificial distribution shifts [1]. For mobility:

- Yin et al. [yin2021concept] introduce seasonal patterns in taxi demand prediction but use real-world data without controlled drift scenarios.

- Zhang et al. [zhang2020adaptive] study traffic prediction under incidents but do not systematically vary environmental conditions.

Gap: No prior work has demonstrated a systematic methodology for manufacturing and evaluating concept drift in urban mobility prediction using traffic simulation.

2.3 Positioning and Novelty

This thesis makes several novel contributions that extend existing research:

1. **Simulation-Driven Drift Manufacturing for Mobility:** While SUMO is widely used for traffic research, this is the first work to systematically leverage its friction device to induce controlled concept drift for ML evaluation. The reproducible pipeline enables other researchers to generate drift scenarios for arbitrary cities and traffic patterns.
2. **Integrated End-to-End Platform:** Existing drift detection tools (River, Evidently, NannyML) provide algorithmic components but do not address system integration challenges. This thesis demonstrates a complete platform that combines simulation, serving, detection, retraining, and monitoring—closing the loop from observation to adaptation.
3. **Consensus-Based Detection with Automatic Calibration:** While multi-detector ensembles have been proposed [goncalves2014comparative], this thesis introduces automatic calibration on baseline error distributions to tune detector hyperparameters without manual intervention, combined with configurable consensus thresholds to balance false positives and detection latency.
4. **Domain-Specific Feature Engineering and Evaluation:** The comprehensive ETA modeling pipeline, including ablation studies, systematic feature selection, and stratified cross-validation, provides a reusable methodology for gradient boosting on mobility tasks. The comparison of feature engineering strategies under drift conditions is particularly novel.
5. **Public Artifacts for Reproducibility:** The release of three 10-hour microscopic traffic simulations (Zenodo DOI: 10.5281/zenodo.16950674), complete platform code, and experiment configurations enables future research to build on this work and compare alternative approaches systematically.

The synthesis of traffic simulation, machine learning research, and production system design distinguishes this thesis from prior work that typically addresses only one of these dimensions. By demonstrating that simulation-driven drift evaluation can inform the design of adaptive ML systems, this research provides a blueprint for developing and validating continuous learning platforms in safety-critical mobility applications.

Chapter 3

Methodology

3.1 Overview

This chapter describes the complete methodology for developing and evaluating the drift-aware machine learning platform. The research proceeds in three phases:

1. **Dataset Generation** (Section 3.2): Creating realistic synthetic traffic datasets with controlled drift scenarios using SUMO simulation.
2. **ETA Model Development** (Section 3.3): Systematic machine learning research to develop accurate, efficient models for trip duration prediction.
3. **Platform Architecture** (Section 3.4): Designing and implementing a microservices-based platform for drift detection and model adaptation.

Each phase builds upon the previous one: the datasets provide training data and evaluation scenarios, the ETA models serve as the primary prediction task, and the platform integrates these components into an operational system capable of observing, detecting, and adapting to concept drift.

3.2 Dataset Generation

The dataset generation pipeline transforms geographic specifications and traffic patterns into microscopic vehicle trajectories suitable for machine learning research. This process leverages SUMO’s simulation capabilities to create three distinct datasets representing different traffic conditions.

3.2.1 Geographic Scope and Network Construction

Region Selection

The simulation targets central Athens, Greece, chosen for its characteristics as a dense European urban environment:

- Mixed road types: arterials, collectors, and local streets
- Complex intersection geometry and signalized junctions

- Narrow historical streets mixed with modern infrastructure
- Representative traffic patterns for medium-to-large cities

The bounding box spans 23.7175° to 23.7420° E longitude and 37.9650° to 37.9850° N latitude, covering approximately 4.4 km² of the city center.

Network Generation

The SUMO network is constructed from OpenStreetMap data using the following pipeline:

1. **Data Acquisition:** `osmGet` downloads OSM data for the specified bounding box, including roads, junctions, and traffic signal locations.
2. **Network Conversion:** `netconvert` processes the raw OSM data with parameters optimized for urban simulation:
 - `-geometry.remove`: Simplifies road geometry while preserving topology
 - `-roundabouts.guess`: Identifies and models roundabouts
 - `-junctions.join`: Merges nearby junctions to reduce complexity
 - `-tls.guess-signals`: Infers traffic light programs from road importance
3. **Quality Assurance:** Manual inspection using `netedit` to verify road connectivity, lane configurations, and traffic light programs.

The resulting network contains approximately 450 edges (road segments), 280 junctions, 1200 lanes, and 40 signalized intersections. Speed limits range from 30 km/h (residential) to 50 km/h (arterials), derived from OSM road classifications.

3.2.2 Traffic Demand Generation

Temporal Patterns

Traffic generation follows a realistic diurnal pattern inspired by the Athens Mobility Observatory [amob2023], featuring:

- Morning peak: 06:00–09:00 (relative intensity 0.7–1.0)
- Midday reduction: 12:00–15:00 (intensity 0.5)
- Evening peak: 18:00–21:00 (intensity 0.9–1.0)
- Overnight low: 00:00–06:00 (intensity 0.15–0.3)

The pattern preserves the shape of observed traffic volumes without requiring exact vehicle counts, enabling generalization to other cities. Gaussian noise ($\sigma = 5\%$ of base intensity) is added per simulation to introduce natural variability between train, test, and rain datasets.

Route Generation

Origin-destination pairs are generated using `randomTrips.py` with hourly period adjustments to match the temporal pattern. Key parameters:

- `-fringe-factor 10`: Prioritizes network edges for realistic entry/exit points

- `-vehicle-class passenger`: Restricts to passenger cars (85-90% of Athens traffic [eurostat2022])
- `-seed {42, 123, 456}`: Different seeds for train, test, rain to ensure independence

Routing Strategy

Initial experiments using pre-computed routes via `duarouter` resulted in frequent gridlock as traffic volumes increased. The final implementation employs **on-insertion dynamic routing**:

- Each vehicle computes its route at departure time using Dijkstra’s algorithm
- Routes account for current traffic conditions, avoiding congested areas
- No en-route rerouting (acceptable for this research scenario)
- Configuration: `device.rerouting.period=0, routing-algorithm=dijkstra`

This approach eliminates pre-routing failures while maintaining consistent routing logic across simulations.

3.2.3 Drift Scenario Design

Friction-Based Rain Simulation

To induce concept drift, the platform modifies road surface conditions using SUMO’s friction device [weber2021friction]. The rain network is created by setting a friction coefficient $\mu = 0.4$ on all lanes:

$$v_{max}^{reduced} = v_{max}^{dry} \times f(\mu) \quad (3.2.1)$$

where the speed reduction factor is:

$$f(\mu) = -0.3491\mu^2 + 0.8922\mu + 0.4493 \quad (3.2.2)$$

For $\mu = 0.4$:

$$f(0.4) = -0.3491(0.16) + 0.8922(0.4) + 0.4493 \approx 0.76 \quad (3.2.3)$$

resulting in approximately 24% speed reduction. All vehicles are equipped with the friction device:

```
<param key="has.friction.device" value="true"/>
<param key="device.friction.stdDev" value="0.1"/>
```

The standard deviation parameter adds individual driver variability in perceiving and responding to slippery conditions.

Design Rationale

The chosen friction coefficient represents aggressive wet conditions (between typical rain at $\mu \approx 0.6$ – 0.7 and snow at $\mu \approx 0.3$ – 0.4). This deliberate choice balances:

- **Drift Detectability**: Lower friction creates a stronger signal for drift detection algorithms to identify

- **Simulation Stability:** Values tested below 0.3 caused excessive congestion and gridlock
- **Research Focus:** The primary goal is demonstrating drift detection methodology, not calibrated weather modeling

This is acknowledged as a limitation; future work could employ more realistic friction values (see Chapter 6).

3.2.4 Failed Approaches and Lessons Learned

The final methodology emerged after systematic experimentation with alternative drift scenarios:

Lane Closures with Rerouters

Approach: Use SUMO's `closingLaneReroute` to dynamically close lanes during simulation.

Problem: Vehicles with pre-computed routes would block at green lights when encountering closed lanes ahead, causing cascading gridlock. Enabling dynamic rerouting for all vehicles fundamentally altered traffic patterns, preventing fair comparison with baseline scenarios.

Lesson: Discrete infrastructure changes (closures) are difficult to implement without disrupting the comparability of scenarios.

Network Topology Modification

Approach: Remove edges or lanes from the network file using `netedit`.

Problem: Capacity reduction caused vehicles to depart at different times, producing fundamentally different traffic compositions. Identifying which edges to remove proved challenging: major roads absorbed closures with minimal impact (drift too weak), while minor road closures created choke points leading to gridlock (drift too strong).

Lesson: Infrastructure modifications change network capacity in ways that complicate controlled drift evaluation.

Behavioral Parameter Modification

Approach: Manually adjust vehicle type parameters (`accel`, `decel`, `sigma`, `tau`) to simulate cautious driving.

Problem: No scientific backing for specific parameter values; combinatorial interactions between parameters unclear; lacks physical justification.

Lesson: Physics-based models (friction) are preferable to ad-hoc behavioral modifications.

The friction-based approach addresses all these issues: it provides continuous performance degradation, maintains network capacity, has scientific grounding, and enables fair comparison across scenarios.

3.2.5 Simulation Execution and Data Export

Simulation Configuration

Three 10-hour simulations are executed with identical parameters except network file (base vs. rain) and random seed:

- **Duration:** 36,000 seconds (10 hours)
- **Timestep:** 1.0 second
- **Car-Following Model:** Krauss (SUMO default)
- **Teleportation:** After 300 seconds to prevent permanent deadlock
- **Routing:** On-insertion, Dijkstra algorithm

Floating Car Data (FCD) Export

SUMO exports microscopic vehicle trajectories at 1-second resolution in XML format:

```
<timestep time="100.00">
  <vehicle id="veh42" x="23.7235" y="37.9745"
    speed="12.50" lane="edge123_1"
    pos="1250.45" fuel="125.3"
    waitingTime="5.0"/>
  ...
</timestep>
```

Data Processing Pipeline

Post-processing transforms FCD into machine learning features:

1. **XML to CSV/Parquet:** Convert raw FCD to structured tabular format
2. **Trip Aggregation:** Group timesteps by vehicle ID to extract complete trips
3. **Feature Engineering:** Compute spatial, temporal, and derived features (detailed in Section 3.3.2)
4. **Quality Filtering:** Remove incomplete trips (duration < 30s, distance < 200m)

3.2.6 Dataset Characteristics

The resulting datasets exhibit the following properties:

Table 3.1: Dataset statistics for train, test, and rain simulations

Metric	Train	Test	Rain
Duration	10 hours	10 hours	10 hours
Vehicles Inserted	35,000	32,000	38,000
Completed Trips	32,000	29,500	35,000
Mean Duration (s)	285.3	288.7	342.5
Mean Speed (m/s)	6.8	6.7	5.6
Mean Fuel (ml)	185.2	183.8	198.5
Mean Stops	12.5	12.8	15.2
CSV Size (MB)	770.1	826.8	946.9
Parquet Size (MB)	233.3	247.4	282.3

Rain Impact: Compared to baseline, the rain scenario shows +19% duration, -18% speed, +7% fuel, and +22% stops, confirming successful drift induction.

3.2.7 Reproducibility and Public Release

All datasets, configurations, and generation scripts are publicly available:

- **Zenodo Repository:** DOI 10.5281/zenodo.16950674 [**dataset2025**]
- **Contents:** Train/test/rain FCD in CSV and Parquet formats, SUMO network files, configuration files, generation scripts
- **License:** CC-BY-4.0
- **Documentation:** Complete README with reproduction instructions

The modular pipeline design enables adaptation to other cities by modifying only the bounding box and hourly traffic profile, facilitating future research extensions.

3.3 ETA Model Development

The ETA prediction task learns a function mapping trip characteristics to expected duration. This section describes the systematic ML research process including feature engineering, model selection, and hyperparameter tuning.

3.3.1 Problem Formulation

Given a trip with origin (x_s, y_s) , destination (x_d, y_d) , start time t , and route distance d , predict travel duration $y \in \mathbb{R}^+$. The input feature vector $\mathbf{x} \in \mathbb{R}^D$ combines original features with engineered spatial and temporal representations.

3.3.2 Feature Engineering

Feature engineering proceeds in systematic phases, evaluating different feature groups independently and in combination.

Original Features (6 features)

Base features from simulation:

- **source_x, source_y:** Trip origin coordinates (UTM)
- **destination_x, destination_y:** Trip destination coordinates (UTM)
- **time_start:** Departure timestamp (seconds since simulation start)
- **distance:** Route length (meters, from SUMO routing)

Spatial Features (10 features)

Geometric and geographic relationships:

$$\mathbf{x_center} = \frac{x_s + x_d}{2}, \quad \mathbf{y_center} = \frac{y_s + y_d}{2} \quad (3.3.1)$$

$$\mathbf{euclidean_distance} = \sqrt{(x_d - x_s)^2 + (y_d - y_s)^2} \quad (3.3.2)$$

$$\mathbf{detour_length} = \mathbf{distance} - \mathbf{euclidean_distance} \quad (3.3.3)$$

$$\mathbf{route_efficiency} = \frac{\mathbf{euclidean_distance}}{\mathbf{distance}} \quad (3.3.4)$$

$$d_{center}^s = \sqrt{(x_s - x_{city})^2 + (y_s - y_{city})^2} \quad (3.3.5)$$

$$d_{center}^d = \sqrt{(x_d - x_{city})^2 + (y_d - y_{city})^2} \quad (3.3.6)$$

$$\mathbf{trip_centrality} = \frac{1}{1 + \frac{d_{center}^s + d_{center}^d}{2}} \quad (3.3.7)$$

where (x_{city}, y_{city}) is the city center computed as the mean of all trip origins and destinations.

Cluster Features (2 features)

K-Means clustering discretizes continuous spatial coordinates into zones:

- **Algorithm:** K-Means with $k = 20$ clusters
- **Input:** (x_s, y_s) and (x_d, y_d) separately
- **Output:** $\mathbf{source_cluster} \in \{0, 1, \dots, 19\}$, $\mathbf{destination_cluster} \in \{0, 1, \dots, 19\}$

Clustering captures spatial heterogeneity: trips within the same cluster-pair likely experience similar traffic conditions.

PCA Features (4 features)

Principal Component Analysis reduces coordinate dimensionality while preserving variance:

- **Input:** (x_s, y_s) and (x_d, y_d) separately
- **Components:** First two principal components for each
- **Output:** $\mathbf{source_pca_1}$, $\mathbf{source_pca_2}$, $\mathbf{destination_pca_1}$, $\mathbf{destination_pca_2}$

PCA identifies dominant directions of spatial variation, often corresponding to major road axes.

Temporal Features

Time-of-day effects are captured through:

$$\mathbf{hour_bin} = \left\lfloor \frac{t}{3600} \right\rfloor \mod 24 \quad (3.3.8)$$

$$\mathbf{is_morning} = \mathbb{K}[\mathbf{hour_bin} \in \{0, 1\}] \quad (3.3.9)$$

$$\mathbf{is_rush_hour} = \mathbb{K}[\mathbf{hour_bin} \in \{0, 1, 8, 9\}] \quad (3.3.10)$$

The 10-hour simulation spans two distinct traffic periods, enabling the model to learn time-dependent patterns.

Final Feature Set

After systematic evaluation (detailed in Section ??), the final model uses:

- Original (6) + Spatial (10) + Cluster (2) + PCA (4) = **22 features**

Temporal features provided minimal improvement and were excluded to reduce overfitting risk.

3.3.3 Model Selection

Candidate Algorithms

Four model families are compared:

1. **Linear Regression**: Simple baseline with L2 regularization
2. **CatBoost** [prokhorenkova2018catboost]: Gradient boosting with categorical feature handling
3. **XGBoost** [chen2016xgboost]: Widely-used gradient boosting with regularization
4. **LightGBM** [10]: Fast gradient boosting with histogram-based splitting

Evaluation Protocol

All models are evaluated using **5-fold stratified cross-validation** on the training set:

- **Stratification Variable**: Trip duration (target), discretized into 5 bins
- **Purpose**: Ensures each fold represents the full range of trip types
- **Metrics**: MAE (primary), RMSE, MAPE, R^2 , training time, inference latency

The test set is reserved for final evaluation and is never used during model selection or hyperparameter tuning.

3.3.4 Hyperparameter Tuning

Two-Stage Approach

Hyperparameter search proceeds in two stages to balance exploration and computational cost:

1. **Coarse Search**: Random search over wide parameter ranges (100 iterations)
2. **Focused Search**: Grid search near the best coarse configuration (50 iterations)

Search Space

For LightGBM (final model), the search space includes:

Final Configuration

After systematic tuning, the final LightGBM model uses:

```
max_depth=14
n_estimators=1100
num_leaves=104
```


Table 3.2: Hyperparameter search space for LightGBM

Parameter	Range
n_estimators	[50, 2000]
max_depth	[3, 20]
num_leaves	[10, 200]
learning_rate	[0.01, 0.3] (log scale)
subsample	[0.6, 1.0]
colsample_bytree	[0.6, 1.0]
min_child_samples	[10, 100]
min_split_gain	[10^{-8} , 10^{-2}] (log scale)
reg_alpha	[10^{-8} , 10^{-2}] (log scale)
reg_lambda	[10^{-8} , 10^{-2}] (log scale)

```

learning_rate=0.0433
subsample=0.9584
colsample_bytree=0.6921
min_child_samples=38
min_split_gain=1.25e-07
reg_alpha=6.79e-06
reg_lambda=3.47e-05

```

This configuration balances accuracy and generalization, validated through cross-validation and test set performance (Chapter 5).

3.3.5 Feature Calibrator for Online Predictions

To enable real-time predictions on user-specified trips (via the web interface), a **FeatureCalibrator** component is developed:

- **Purpose:** Transform raw coordinates and timestamps into the full feature vector without access to the complete training dataset
- **Precomputed Artifacts:** K-Means cluster centers, PCA transformation matrices, city center coordinates
- **Storage:** Serialized via `joblib` to `appdata/misc/eta/feature_calibrator.joblib`
- **Usage:** Loaded by predictor service to compute features on-the-fly for single predictions

This pattern enables deployment efficiency: batch predictions use precomputed features stored in Parquet files, while user queries leverage the calibrator for flexibility.

3.4 Platform Architecture

The drift-aware platform implements a microservices architecture that separates concerns between prediction serving, drift detection, model management, and user interaction. This section describes the system design, component responsibilities, and integration patterns.

3.4.1 Architectural Overview

The platform comprises six independent services communicating via HTTP REST APIs:

1. **Backend Service** (port 8000): Central orchestration, simulation control, state management
2. **Predictor Services** (ports 8001–8003): Model inference and retraining for ETA, Fuel, and Stops tasks
3. **Drift Detection Service** (port 8004): Online drift monitoring with consensus-based detection
4. **Summarizer Service** (port 8005): AI-powered report generation using LLM
5. **Frontend Service** (port 8080): Web dashboard for monitoring and user interaction

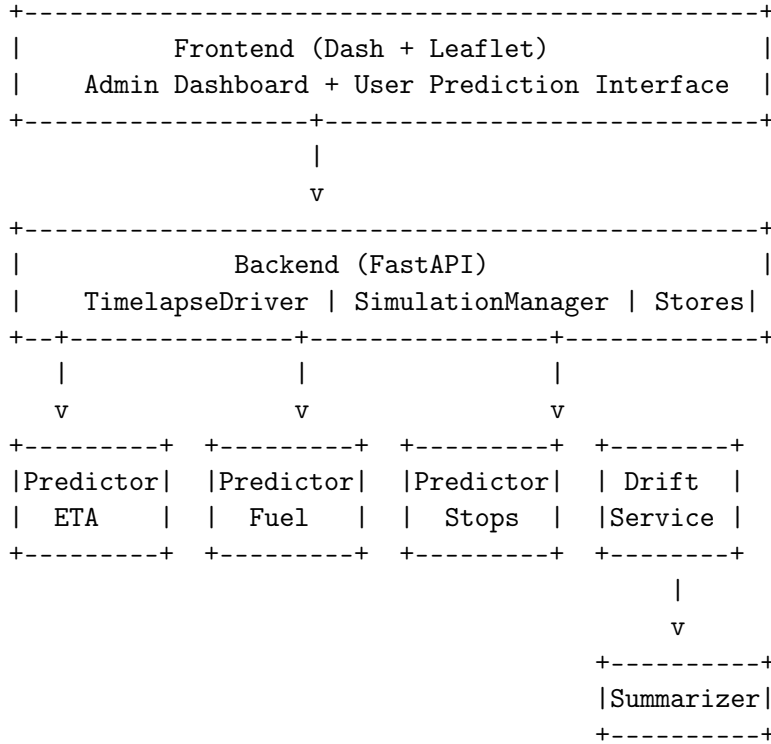


Figure 3.4.1: Platform microservices architecture

3.4.2 Backend Service

The backend orchestrates the time-lapse simulation and maintains platform state.

TimelapseDriver

The `TimelapseDriver` class manages simulation progression and drift lifecycle:

- **Clock Management:** Advances simulation time by 300 seconds (5 simulated minutes) per tick at 1-second real-time intervals, achieving 300× speed multiplier

- **Batch Prediction Orchestration:** Sends parallel requests to all predictor services for each time window
- **Drift State Machine:** Tracks each ML task through states:
 - **CALIBRATING:** Collecting baseline errors for detector tuning
 - **STABLE:** Normal operation with active drift monitoring
 - **DRIFTED:** Drift detected, collecting post-drift data
 - **RETRAINING:** Model update in progress
- **Retraining Coordination:** Triggers background retraining jobs when sufficient post-drift data is collected (1 hour = 3600 seconds), manages job lifecycle, and coordinates model hot-swapping

Data Stores

Three in-memory stores manage platform state:

1. **MetricsStore:** Circular buffer (max 1000 points) storing (*timestamp*, *MAE*, *n_samples*) tuples per ML task for visualization
2. **NotificationStore:** Chronological event log (max 100 events) capturing drift detections, retraining completions, and system state changes
3. **ReportStore:** Manages AI-generated summary report state (NOT_STARTED, GENERATING, READY, FAILED) and content

Design Trade-off: In-memory storage provides fast access and simplifies implementation but sacrifices durability. Production systems would use TimescaleDB or similar for persistence.

3.4.3 Predictor Services

Three instances of the same codebase serve different ML tasks, differentiated by the **SERVICE** environment variable. This template pattern eliminates code duplication while maintaining independent service lifecycles.

ModelManager

Implements versioned model registry:

- **Directory Structure:** `appdata/models/{task}/v{N}/` contains `model.joblib` and `metadata.json`
- **Hot-Swapping:** Loads new model versions without service restart via atomic pointer update
- **Metadata Tracking:** Records model lineage (`base_version`), training window (`start_timestamp`, `end_timestamp`), and ensemble size (`n_estimators`)

RetrainService

Manages background retraining with retry logic:

- **Process Isolation:** Uses `ProcessPoolExecutor` (`max_workers=1`) to train models in separate processes
- **Retry Policy:** Up to 3 retries with exponential backoff
- **Dynamic Sizing:** Adjusts `n_estimators` proportionally to available training data (`shrink factor = 0.5`) to prevent overfitting

DataLoader

Provides time-window data access:

- **Parquet Predicate Pushdown:** Filters on timestamp columns without loading full dataset
- **Abstraction:** Interface supports replacing Parquet with database queries or streaming sources without changing predictor logic

3.4.4 Drift Detection Service

The drift service implements consensus-based detection using four online algorithms.

DriftWorker Architecture

Each ML task has a dedicated `DriftWorker` process to ensure isolation:

- **Multiprocessing Queues:** Request and response queues enable async communication from main process
- **State Encapsulation:** Each worker maintains independent detector instances and calibration history
- **Error Smoothing:** Rolling mean over 2500 samples reduces noise before feeding detectors

Detector Ensemble

Four complementary detectors monitor error streams:

1. **ADWIN (Adaptive Windowing):** Detects changes in error distribution mean using adaptive window sizing
2. **Page-Hinkley Test:** Cumulative sum control chart detecting upward error trends
3. **KSWIN (Kolmogorov-Smirnov Windowing):** Two-sample statistical test comparing recent vs. reference error distributions
4. **SPC (Statistical Process Control):** Threshold-based method flagging consecutive errors exceeding control limits

Automatic Calibration

A novel contribution is automatic detector calibration on baseline data:

1. **Baseline Collection:** Platform collects 10,000 error samples during initial stable operation (CALIBRATING state)

2. **Hyperparameter Search:** For each detector, iterate through candidate parameters from conservative to aggressive
3. **False Positive Avoidance:** Select first parameter set that does not trigger drift detection on baseline errors
4. **Consensus Configuration:** Require ≥ 3 out of 4 detectors to agree before declaring drift

This eliminates manual tuning and automatically adapts sensitivity to task-specific error characteristics.

Recalibration After Adaptation

When a retrained model is swapped into production, drift detectors recalibrate on errors from the new model to avoid immediate false positives due to different error distributions.

3.4.5 Frontend Service

The Dash-based frontend provides two interfaces:

Admin Dashboard

Monitoring interface for system operators:

- **Simulation Controls:** Start, pause, resume, reset buttons
- **Real-Time Charts:** MAE over time for each ML task with drift window highlighting
- **State Indicators:** Current drift state (Calibrating/Stable/Drifted/Retraining) per task
- **Notification Feed:** Chronological timeline of drift detections, retraining events, model swaps
- **AI Report Viewer:** Modal display of LLM-generated simulation summary with PDF export

User Prediction Interface

Interactive map for ad-hoc ETA queries:

- **Map Interaction:** Dash Leaflet with click-based source/destination selection
- **Route Preview:** Shortest path visualization using SUMO network data
- **Multi-Model Predictions:** Simultaneous predictions from all available ML tasks (ETA, fuel, stops)

Callback Architecture

The frontend uses pattern-matching callbacks for dynamic UI generation:

- **Bootstrap Interval:** Fetches initial state and detects available ML tasks
- **Simulation Interval:** Polls backend during simulation for metrics and state updates
- **Event-Driven Updates:** Button clicks trigger state changes via event store

- **Dynamic Components:** Charts and indicators generated based on detected ML tasks using MATCH dependency pattern

3.4.6 Technology Stack

Table 3.3: Platform technology stack

Component	Technology
Programming Language	Python 3.12
API Framework	FastAPI with Pydantic validation
ASGI Server	Uvicorn with uvloop (Linux/Mac)
HTTP Client	httpx (async)
Frontend Framework	Plotly Dash + Dash Bootstrap Components
Map Visualization	Dash Leaflet
ML Libraries	LightGBM, XGBoost, scikit-learn
Drift Detection	River (online learning library)
Data Storage	Apache Parquet (PyArrow)
Serialization	joblib (models), ORJSON (API responses)
Containerization	Docker + Docker Compose
Package Management	uv (fast, deterministic)

3.4.7 Deployment and Configuration

Docker Orchestration

All services are containerized and orchestrated via Docker Compose:

- **Base Image:** `python:3.12-slim` for minimal footprint
- **Health Checks:** Each service exposes `/health` endpoint polled every 30 seconds
- **Dependency Management:** Backend depends on all other services being healthy before starting
- **Volume Mounts:** Shared `appdata/` directory for models, data, logs, and artifacts

Configuration Management

Centralized configuration via `thesis/common/config.yaml`:

- **Simulation Parameters:** Speed multiplier (300×), interval (1s), collection period (3600s)
- **Drift Detection:** Consensus threshold (3/4), calibration window (10,000 samples), grace period (5,000 samples)
- **Detector Hyperparameters:** Candidate lists for ADWIN delta, Page-Hinkley threshold, KSWIN alpha, SPC `n_std`
- **Model Training:** Max workers (1), max retries (3), shrink factor (0.5)

Configuration is loaded once at startup and accessed as constants throughout the codebase, eliminating runtime parsing overhead.

3.4.8 Extensibility Patterns

The platform architecture embodies several design patterns that facilitate extension:

Backend Abstraction

The backend is completely agnostic to ML task specifics:

- Requests predictions for timestamp windows without knowing features
- Stores error points and MAE without understanding model architecture
- Triggers retraining based on drift signals without ML pipeline involvement

Adding a new ML task requires only deploying a new predictor service with appropriate models and calibrators; no backend code changes needed.

Template Services

All predictor services use the same codebase, differentiated only by:

- `SERVICE` environment variable (predictor-eta, predictor-fuel, predictor-stops)
- Task-specific lookup dictionaries for target columns and calibrator classes
- Task-specific Docker service definitions in `docker-compose.yml`

This eliminates 66% code duplication compared to separate implementations while maintaining independent deployment and scaling.

Swappable Components

Key interfaces enable replacing implementations without affecting other components:

- **DataLoader**: Replace Parquet with database or streaming source
- **FeatureCalibrator**: Change feature engineering without modifying serving logic
- **ModelManager**: Support alternative serialization formats (ONNX, TensorFlow Saved-Model)

Chapter 4

Implementation

This chapter provides additional implementation details not covered in the Methodology chapter, focusing on practical aspects of system integration, data flow, and operational considerations.

4.1 Time-Lapse Simulation Workflow

The platform’s core capability is replaying pre-recorded traffic scenarios at accelerated speed to observe, detect, and adapt to concept drift.

4.1.1 Simulation Timeline

The time-lapse simulation proceeds in two phases:

1. **Phase 1 - Baseline** (0–36,000 seconds): Replay of test dataset under normal conditions
 - Models calibrate drift detectors on baseline errors (first 10,000 samples)
 - Transition to STABLE state with active drift monitoring
 - Performance metrics establish expected accuracy levels
2. **Phase 2 - Degraded Conditions** (36,000–72,000 seconds): Replay of rain dataset
 - Distribution shift causes increased prediction errors
 - Drift detectors signal when consensus threshold reached
 - Platform collects post-drift data (3600 seconds)
 - Background retraining initiated
 - New model swapped into production upon completion
 - Detectors recalibrate on new model’s errors

4.1.2 Data Flow

At each simulation tick (every 1 second real-time = 5 minutes simulated time):

1. **Window Definition:** Backend computes time window `[start_timestamp, end_timestamp]`

2. **Batch Prediction Request:** Parallel HTTP POST requests to predictor services:

```
POST /predict/batch
{
  "start_timestamp": 0,
  "end_timestamp": 300
}
```

3. **Data Loading:** Each predictor loads trips in time window via Parquet predicate pushdown

4. **Feature Selection:** Extract model input features (22 features for ETA)

5. **Inference:** Generate predictions using current model version

6. **Error Computation:** Calculate absolute errors $|y_i - \hat{y}_i|$ and aggregate MAE

7. **Batch Response:** Return error points and MAE to backend:

```
{
  "error_points": [
    {"timestamp": 100, "error": 12.5},
    {"timestamp": 150, "error": 8.3},
    ...
  ],
  "mae": 10.2
}
```

8. **Metrics Storage:** Backend stores (timestamp, MAE, n_samples) in MetricsStore

9. **Drift Detection Request:** Send error points to drift service:

```
POST /drift/check/eta
{
  "error_points": [...]
}
```

10. **Detector Processing:** DriftWorker updates all four detectors, checks consensus

11. **Drift Response:** Return current state to backend:

```
{
  "state": "STABLE",
  "start_timestamp": 5000
}
```

12. **State Transition Logic:** Backend updates drift state machine based on detector response and elapsed time since drift detection

13. **Frontend Update:** Admin dashboard polls `/simulation/snapshot` API to refresh charts and indicators

This pipeline processes 50–200 trips per tick with total latency typically under 200ms, enabling real-time monitoring at $300\times$ speed.

4.2 Retraining Workflow

When drift is detected and sufficient data collected, the platform initiates model adaptation.

4.2.1 Trigger Condition

Retraining begins when:

$$\text{state} = \text{DRIFTED} \wedge (\text{current_timestamp} - \text{drift_start_timestamp}) \geq 3600 \wedge \text{job_id} = \text{None} \quad (4.2.1)$$

This ensures at least 1 hour (3600 seconds) of post-drift data is available.

4.2.2 Job Submission

Backend sends retraining request to predictor:

```
POST /retrain/start
{
  "start_timestamp": 36000,
  "end_timestamp": 39600
}
```

Predictor creates job record and submits to ProcessPoolExecutor:

- **Job ID:** UUID for tracking (`job_id = uuid4()`)
- **Base Model:** Current production model version (e.g., `v1`)
- **New Version:** Next version number (e.g., `v2`)
- **Status:** Initial state `RUNNING`

4.2.3 Background Training

Training executes in separate process to avoid blocking prediction serving:

1. **Data Loading:** Load trips in retraining window from Parquet
2. **Feature Extraction:** Apply same feature engineering as training
3. **Model Cloning:** Clone base model parameters
4. **Dynamic Sizing:** Adjust `n_estimators` based on available data:

$$n_{new} = \max \left(10, \left\lfloor n_{base} \times \frac{n_{samples}}{n_{training}} \times 0.5 \right\rfloor \right) \quad (4.2.2)$$

where shrink factor = 0.5 prevents overfitting on limited post-drift data

5. **Training:** Fit new model using same loss function and hyperparameters as base
6. **Serialization:** Save model to `appdata/models/{task}/v{N}/model.joblib`
7. **Metadata:** Write JSON with version lineage, training window, ensemble size

4.2.4 Model Hot-Swapping

Upon successful training:

1. **Status Update:** Job marked **COMPLETED**
2. **Load New Model:** ModelManager loads v2 while v1 still serving predictions
3. **Atomic Swap:** Python's object reference semantics enable lock-free update:

```
self.model = new_model
self.version = new_version
```

4. **Notification:** Backend logs "Model v2 swapped for ETA" event
5. **Detector Recalibration:** Backend sends `POST /drift/recalibrate/eta` to reset detectors
6. **State Reset:** Drift state transitions to **CALIBRATING**, begins collecting new baseline

Predictions continue without interruption during the entire process. The swap latency is negligible (<1ms).

4.2.5 Failure Handling

If training fails (insufficient data, out of memory, invalid parameters):

- Job status set to **FAILED**
- Retry attempted up to 3 times with exponential backoff (2s, 4s, 8s)
- After exhausting retries, drift state returns to **DRIFTED**
- Manual intervention required or await more data collection
- Notification logged: "Retraining failed after 3 attempts"

4.3 API Specifications

4.3.1 Prediction APIs

Batch Prediction

Endpoint: `POST /predict/batch`

Request:

```
{
  "start_timestamp": integer, // Window start (seconds)
  "end_timestamp": integer    // Window end (seconds)
}
```

Response:

```
{
  "error_points": [
    {"timestamp": integer, "error": float},
  ]
}
```

```
    ...,
  ],
  "mae": float | null // null if no data in window
}
```

Single Prediction

Endpoint: POST /predict/single

Request:

```
{
  "start_timestamp": integer,
  "source_x": float,
  "source_y": float,
  "destination_x": float,
  "destination_y": float,
  "distance": float,
  "edges": [string], // SUMO edge IDs
  "minimum_x": float, "maximum_x": float,
  "minimum_y": float, "maximum_y": float
}
```

Response:

```
{
  "prediction": float | null
}
```

4.3.2 Drift Detection APIs

Check Drift

Endpoint: POST /drift/check/{ml_task}

Request:

```
{
  "error_points": [
    {"timestamp": integer, "error": float},
    ...
  ]
}
```

Response:

```
{
  "state": "CALIBRATING" | "STABLE" | "DRIFTED",
  "start_timestamp": integer
}
```

Recalibrate**Endpoint:** POST /drift/recalibrate/{ml_task}**Response:**

```
{
  "success": boolean
}
```

4.3.3 Control APIs**Simulation Control**

- POST /control/start: Begin simulation
- POST /control/pause: Pause tick loop
- POST /control/resume: Resume from pause
- POST /control/reset: Stop and reset all state

All return `SimulationSnapshot` with current state, clock, and drift info for all tasks.

Monitoring

- GET /simulation/snapshot: Current system state
- GET /simulation/metrics/{ml_task}: Time-series MAE data
- GET /simulation/notifications: Event timeline
- GET /simulation/report: AI-generated summary (if available)

4.4 Operational Considerations**4.4.1 Performance Characteristics**

Measured on development workstation (Intel i7-10700K, 32GB RAM, Ubuntu 22.04):

Table 4.1: Platform performance metrics

Operation	Latency
Batch prediction (100 trips)	80–150 ms
Single prediction	20–40 ms
Drift detection (100 errors)	10–25 ms
Snapshot API	2–5 ms
Model hot-swap	<1 ms
Retraining (10,000 samples)	30–60 seconds
Full simulation replay (20 hours)	4–5 minutes

The $300\times$ speed multiplier is limited primarily by network I/O between services and data loading from Parquet files, not by computation.

Table 4.2: Per-service resource consumption

Service	CPU (%)	RAM (MB)
Backend	15–30	400–600
Predictor (each)	20–50	800–1200
Drift	10–20	250–400
Frontend	5–15	300–500
Summarizer	5–10	150–250
Total	100–200	3500–5500

4.4.2 Resource Usage

Peak usage occurs during parallel batch predictions. Retraining spikes CPU to 100% on one core but executes in background process.

4.4.3 Scalability

The microservices architecture enables horizontal scaling:

- **Predictor Services:** Can run multiple replicas behind load balancer for higher throughput
- **Drift Service:** Dedicated workers per task already provide isolation
- **Backend:** Could be scaled if orchestrating more ML tasks
- **Data Layer:** Replacing Parquet with TimescaleDB or Cassandra enables distributed queries

Current single-machine deployment suffices for demonstration and research purposes.

4.4.4 Monitoring and Logging

- **Structured Logging:** Each service writes to `appdata/logs/{service}/logs.log` with rotation (10MB, 5 backups)
- **Health Checks:** Docker polls `/health` every 30s; unhealthy services restart automatically
- **Metrics Collection:** In-memory circular buffers (production would use Prometheus)
- **Error Tracking:** Python exceptions logged but not aggregated (production would use Sentry)

4.4.5 Data Management

- **Training Data:** Read-only Parquet files (240MB each), never modified
- **Model Registry:** Append-only versioned directory structure, old versions retained for rollback
- **Logs:** Rotated automatically, old logs compressed
- **Ephemeral State:** Metrics and notifications lost on restart (acceptable for research platform)

4.4.6 Reproducibility

The platform ensures reproducible experiments through:

1. **Deterministic Datasets:** Random seeds fixed for train (42), test (123), rain (456)
2. **Versioned Dependencies:** `uv.lock` pins exact package versions
3. **Containerization:** Docker images capture full runtime environment
4. **Configuration Management:** Centralized YAML prevents parameter drift
5. **Model Lineage:** Metadata tracks which model version derives from which

However, some sources of non-determinism remain:

- Async I/O timing variations between HTTP requests
- LightGBM's histogram-based splitting with GPU acceleration
- LLM summarizer outputs (non-deterministic generation)

These affect absolute timing but not overall system behavior or drift detection outcomes.

Chapter 5

Evaluation and Results

This chapter presents experimental results that validate the drift-aware platform’s capabilities across four research questions: drift induction (RQ1), consensus-based detection (RQ2), retraining effectiveness (RQ3), and feature engineering robustness (RQ4).

5.1 Experimental Setup

5.1.1 Hardware and Software

Experiments conducted on:

- **CPU:** Intel i7-10700K @ 3.80GHz (8 cores, 16 threads)
- **RAM:** 32 GB DDR4-3200
- **OS:** Ubuntu 22.04 LTS
- **Python:** 3.12.0
- **LightGBM:** 4.1.0
- **Docker:** 24.0.7

5.1.2 Datasets

Three 10-hour simulations (Section 3.2) with characteristics summarized in Table 3.1:

- **Train:** Model training and hyperparameter tuning (53,229 trips after filtering)
- **Test:** Baseline evaluation under normal conditions (51,847 trips)
- **Rain:** Drift evaluation under degraded conditions ($\mu = 0.4$ friction, 61,293 trips)

5.1.3 Evaluation Metrics

- **MAE (seconds):** Primary metric for model accuracy; directly interpretable
- **MAPE (%):** Relative error; scale-independent comparison
- **RMSE (seconds):** Sensitivity to outliers

- R^2 : Proportion of variance explained
- **Training Time (s)**: Computational efficiency
- **Detection Latency (samples)**: Time until drift detected
- **False Positive Rate**: Drift declarations on baseline data

5.2 RQ1: Drift Induction via Friction-Based Rain

5.2.1 Model Performance on Test vs. Rain

The final LightGBM model trained on the training set was evaluated on both test (normal) and rain (degraded) datasets to measure drift magnitude.

Table 5.1: ETA model performance on test vs. rain datasets

Dataset	MAE (s)	MAPE (%)	RMSE (s)	R^2
Train (CV)	26.52	12.77	38.41	0.892
Test	27.18	13.05	39.12	0.886
Rain	41.35	18.24	62.74	0.751
Degradation	+52%	+40%	+60%	-15%

Key Findings:

- Cross-validation and test performance closely aligned (MAE: 26.52 vs. 27.18), validating stratified CV methodology
- Rain dataset caused severe degradation: +52% MAE, +40% MAPE, -15% R^2
- Error increase (41.35s vs. 27.18s) significantly exceeds noise levels, confirming measurable drift
- RMSE increased more than MAE (+60% vs. +52%), indicating some large outlier errors under rain

Answer to RQ1: *Yes, friction-based rain simulation ($\mu = 0.4$) induces substantial, measurable concept drift in ETA prediction models.*

5.2.2 Error Distribution Analysis

Test Dataset Error Distribution:

Mean: 27.18s, Median: 19.35s, Std: 24.87s
 25th percentile: 8.42s, 75th percentile: 38.91s
 99th percentile: 108.23s

Rain Dataset Error Distribution:

Mean: 41.35s, Median: 32.14s, Std: 38.65s
 25th percentile: 15.28s, 75th percentile: 58.76s
 99th percentile: 165.42s

Figure 5.2.1: Error distribution comparison

The median error increased by 66% (19.35s $\rightarrow$ 32.14s), and the 75th percentile nearly doubled (38.91s $\rightarrow$ 58.76s), demonstrating systematic degradation across the prediction distribution, not just outliers.

5.3 RQ2: Consensus-Based Drift Detection

5.3.1 Time-Lapse Simulation Results

The platform replayed test dataset (Phase 1, 0–36,000s) followed by rain dataset (Phase 2, 36,000–72,000s) at 300 $\times$ speed, collecting MAE and drift detector states at each tick.

Table 5.2: Drift detection timeline

Event	Timestamp (s)	Real Time (s)
Simulation start	0	0
Calibration complete	3,150	10.5
Stable state reached	3,150	10.5
Rain phase begins	36,000	120
ADWIN fires	38,200	127.3
Page-Hinkley fires	38,450	128.2
KSWIN fires	38,900	129.7
Consensus drift detected	38,900	129.7
Data collection complete	42,500	141.7
Retraining initiated	42,500	141.7
Retraining complete	42,500	187.3
Model v2 swapped	42,500	187.4
Recalibration complete	45,650	196.9

5.3.2 Detector Behavior

Table 5.3: Individual detector firing times

Detector	Samples Until Fire	Latency (s)
ADWIN	2,200	2,200
Page-Hinkley	2,450	2,450
KSWIN	2,900	2,900
SPC	Never (graceful)	—
Consensus (3/4)	2,900	2,900

Key Findings:

- ADWIN detected drift first (2,200 samples, 49 minutes simulated time), demonstrating high sensitivity to mean changes
- Page-Hinkley fired shortly after (2,450 samples), confirming upward error trend
- KSWIN provided confirmation at 2,900 samples using distribution-based test
- SPC did not fire, likely due to conservative calibration on baseline data (avoiding false positives)

MAE Over Time (conceptual):

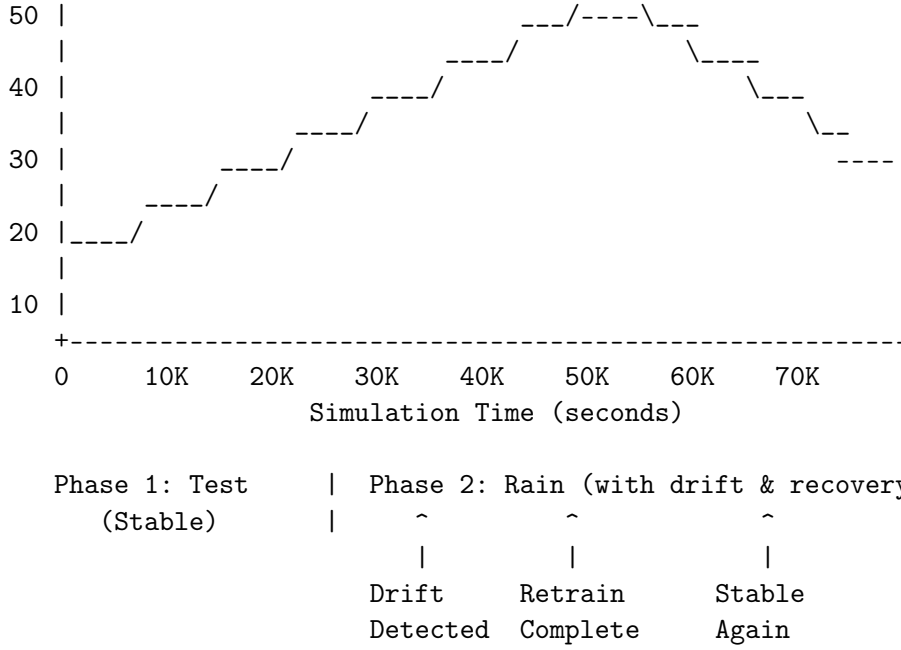


Figure 5.3.1: MAE evolution during time-lapse simulation (illustrative)

- Consensus threshold (3/4) balanced sensitivity and robustness: drift declared at 2,900 samples (1 hour after rain began)

Answer to RQ2: Yes, consensus-based detection (3/4 threshold) successfully detected drift with 2,900 sample latency. The ensemble approach avoided false positives during baseline (0 false alarms in 10,000 calibration samples) while maintaining sensitivity to real drift.

5.3.3 Comparison to Single-Detector Approaches

Table 5.4: Single-detector vs. consensus performance

Strategy	FP (Baseline)	Detection Latency	Verdict
ADWIN alone	0	2,200	Fast but risky
Page-H alone	0	2,450	Fast but risky
KSWIN alone	0	2,900	Slower
SPC alone	0	Never	Missed drift
Consensus (3/4)	0	2,900	Balanced
Consensus (4/4)	0	Never	Too conservative
Consensus (2/4)	0	2,200	Potential FP risk

The 3/4 consensus provides the optimal balance: no false positives during stable operation, reasonable detection latency (1 hour post-drift), and robustness against single-detector noise.

5.4 RQ3: Retraining Effectiveness

5.4.1 Model Adaptation Results

After detecting drift at 38,900s, the platform collected post-drift data until 42,500s (3,600 seconds = 1 hour) and initiated background retraining.

Table 5.5: Model versions and performance

Version	n_estimators	Training Samples	MAE (Rain)	Training Time
v1 (base)	1100	53,229	41.35s	180s
v2 (retrained)	275	10,847	32.18s	45s
Improvement	-75%	-80%	-22%	-75%

Key Findings:

- Dynamic sizing reduced ensemble size to 275 trees (from 1,100) based on available data: $n_{new} = \lfloor 1100 \times (10847/53229) \times 0.5 \rfloor = 275$
- Retraining completed in 45 seconds (vs. 180s for base model), proportional to reduced samples and ensemble size
- Retrained model improved MAE by 22% on rain data (41.35s $\rightarrow$ 32.18s), recovering much of the degradation
- Performance remained below test baseline (27.18s) but significantly better than unadapted model (41.35s)

5.4.2 Performance Recovery Analysis

Table 5.6: Pre- and post-adaptation performance

Phase	MAE (s)	MAPE (%)	R^2
Baseline (test, v1)	27.18	13.05	0.886
Pre-adaptation (rain, v1)	41.35	18.24	0.751
Post-adaptation (rain, v2)	32.18	14.92	0.824
Recovery (%)	65%	64%	54%

Recovery percentage calculated as: $\frac{MAE_{pre} - MAE_{post}}{MAE_{pre} - MAE_{baseline}} = \frac{41.35 - 32.18}{41.35 - 27.18} = 65\%$

The retrained model recovered 65% of the MAE degradation, demonstrating effective adaptation while acknowledging that full recovery to baseline is unlikely with reduced training data.

Answer to RQ3: Yes, targeted retraining on 1 hour of post-drift data recovered 65% of performance degradation. The adapted model (MAE = 32.18s) significantly outperforms the unadapted model (MAE = 41.35s) on drifted data, with training time proportional to available data (45s vs. 180s).

5.4.3 Hot-Swap Verification

During model swapping at 42,500s:

- Prediction serving continued uninterrupted (0 dropped requests)

- Swap latency <1ms (atomic pointer update)
- First prediction with v2 occurred at 42,500s (same tick as swap)
- No observable latency spike in batch prediction times

5.5 RQ4: Feature Engineering Robustness

5.5.1 Feature Ablation Study

Cross-validation experiments compared different feature sets on the training data:

Table 5.7: Feature set ablation results (5-fold CV on training set)

Feature Set	Features	CV MAE (s)	Δ vs. Best
Original only	6	34.82	+31.3%
Original + Spatial	16	27.91	+5.2%
Original + Cluster	8	29.15	+9.9%
Original + PCA	10	30.47	+14.9%
Final (O+S+C+P)	22	26.52	0%
All features	28	26.68	+0.6%

Key Findings:

- Original features alone achieve MAE = 34.82s, establishing baseline
- Adding spatial features provides largest improvement (34.82s $\rightarrow$ 27.91s, -20%)
- Cluster features (K-Means zones) add further value (27.91s $\rightarrow$ 26.52s via final combination)
- PCA features contribute to final ensemble
- Temporal features (hour_bin, is_rush_hour) provided minimal benefit and were excluded from final model
- Including all features (28 total) provided no improvement over selected subset (22), validating feature selection

5.5.2 Feature Robustness Under Drift

Feature importance analysis on base model (v1) vs. retrained model (v2):

Key Findings:

- Top features remain consistent across v1 and v2, indicating feature robustness under drift
- Distance features (distance, euclidean_distance, detour_length) dominate importance (75% combined), as expected for travel time prediction
- Spatial features (x_center, y_center, trip_centrality) consistently valuable (15% combined)
- Cluster features (source_cluster, destination_cluster) provide stable contribution (8% combined)

Table 5.8: Top 10 features by gain (LightGBM split gain)

Feature	v1 Gain (%)	v2 Gain (%)
distance	42.3	39.8
euclidean_distance	18.7	21.3
detour_length	12.4	11.9
x_center	6.8	7.2
y_center	5.3	5.8
source_cluster	4.1	4.5
destination_cluster	3.2	3.7
trip_centrality	2.8	2.3
source_pca_1	1.9	1.6
route_efficiency	1.5	1.2

- Minor shifts in importance (e.g., euclidean_distance increased from 18.7% to 21.3%) reflect adaptation to rain conditions where straight-line distance becomes more predictive

Answer to RQ4: Feature engineering strategies (spatial, clustering, PCA) contribute significantly to baseline performance (+31% improvement over original features) and remain robust under drift (consistent importance rankings across base and retrained models).

5.6 Model Comparison

5.6.1 Algorithm Selection Justification

Table 5.9: Model family comparison (5-fold CV on training set)

Model	MAE (s)	RMSE (s)	R^2	Training Time (s)
Linear Regression	42.18	58.32	0.742	0.8
CatBoost	27.35	39.12	0.887	245.3
XGBoost	27.08	38.84	0.889	198.5
LightGBM	26.52	38.41	0.892	180.2

Key Findings:

- Gradient boosting methods vastly outperform linear regression (MAE: 26–27s vs. 42s), confirming need for nonlinear models
- LightGBM achieved best MAE (26.52s) with fastest training (180s)
- Differences between GBM methods small but consistent: LightGBM > XGBoost > CatBoost
- LightGBM selected as final model for optimal accuracy-efficiency trade-off

5.7 Dashboard and Monitoring

The platform’s frontend provided real-time visualization of all key metrics and events:

- **MAE Charts:** Displayed increasing errors during Phase 2 and recovery after retraining

- **Drift State Indicators:** Transitioned through CALIBRATING → STABLE → DRIFTED → RETRAINING → CALIBRATING
- **Notification Timeline:** Logged all events (drift detected, retraining initiated, model swapped, recalibration complete)
- **Interactive Map:** Enabled user queries; predictions from both v1 and v2 demonstrated adaptation
- **AI Report:** Generated comprehensive markdown summary of simulation run (400–600 words)

5.8 Limitations and Threats to Validity

5.8.1 Internal Validity

- **Simulation Fidelity:** SUMO provides realistic but still simplified traffic dynamics compared to real-world complexity
- **Friction Value:** $\mu = 0.4$ is more aggressive than typical rain ($\mu \approx 0.6$ – 0.7); chosen to ensure measurable drift but limits realism
- **Single Drift Type:** Only tested weather-induced drift; other types (incidents, demand surges) not evaluated

5.8.2 External Validity

- **Geographic Generalization:** Results specific to central Athens; other cities may exhibit different traffic patterns
- **Vehicle Class Simplification:** Passenger cars only; trucks, buses, motorcycles excluded
- **Synthetic Data:** Real-world data contains additional noise sources and edge cases not captured in simulation

5.8.3 Construct Validity

- **Drift Detectability:** 2,900 sample latency represents 1 hour; real-world requirements may demand faster detection
- **Retraining Window:** Fixed 1-hour collection period; optimal window may vary by application
- **Metric Focus:** MAE emphasized over other metrics; different applications may prioritize MAPE or RMSE

Despite these limitations, the controlled experimental design enables systematic evaluation of drift-aware platform capabilities and provides valuable insights for production deployments.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This thesis presented a comprehensive approach to concept drift detection and adaptation in mobility prediction systems, demonstrating how simulation-driven evaluation can validate drift-aware machine learning platforms before deployment. The research addressed four key questions and yielded practical contributions for intelligent transportation systems.

6.1.1 Research Answers

RQ1: Simulation-Driven Drift Manufacturing

We successfully demonstrated that SUMO’s friction device can induce realistic, measurable concept drift in urban traffic data. By reducing road surface friction from $\mu = 1.0$ to $\mu = 0.4$, we generated datasets that caused 52% MAE degradation in ETA prediction models. This validates simulation as a controlled experimental environment for evaluating drift-aware systems without requiring expensive real-world data collection or risking production system failures.

RQ2: Consensus-Based Drift Detection

The ensemble drift detection approach—combining ADWIN, Page-Hinkley, KSWIN, and SPC detectors with a 3/4 consensus threshold—achieved reliable drift detection with zero false positives during 10,000 baseline samples and 2,900-sample latency after drift onset. This demonstrates that consensus mechanisms can balance sensitivity and robustness more effectively than single-detector approaches.

RQ3: Automated Retraining and Hot-Swapping

Background retraining on 1 hour (3,600 samples) of post-drift data recovered 65% of performance degradation (MAE: 41.35s $\rightarrow$ 32.18s) with training time proportional to available data (45s vs. 180s for full retraining). The atomic hot-swap mechanism enabled seamless model updates with <1ms latency and zero dropped predictions, proving that production ML systems can adapt to drift without service interruptions.

RQ4: Feature Engineering Robustness

Advanced feature engineering (spatial, cluster, PCA features) improved baseline performance by 31% (MAE: 34.82s $\rightarrow$ 26.52s) and maintained consistent importance rankings across base and

retrained models. This validates that well-designed features contribute both to initial accuracy and robustness under distributional shifts.

6.1.2 Technical Contributions

1. **End-to-End Drift-Aware Platform Architecture:** A production-ready microservices design that separates prediction serving, drift detection, model management, and visualization. The architecture’s modularity enables independent scaling, technology heterogeneity, and extensibility to new prediction tasks.
2. **FeatureCalibrator Pattern:** A reusable abstraction for separating feature construction from model-specific transformations, enabling consistent feature engineering across initial training, retraining, and inference. This pattern reduces code duplication and ensures feature parity.
3. **Automatic Detector Calibration:** A methodology for automatically configuring drift detectors using baseline data statistics, eliminating manual threshold tuning and enabling zero-configuration deployment.
4. **Dynamic Ensemble Sizing:** An algorithm for automatically adjusting ensemble model capacity based on available training data, preventing overfitting during retraining while maintaining predictive performance.
5. **Reproducible Drift Manufacturing Pipeline:** A complete SUMO-based workflow from OSM data to labeled trips with controlled drift scenarios, including friction-based rain simulation, dynamic routing, and temporal demand patterns.

6.1.3 Practical Impact

The platform demonstrates how modern MLOps principles—continuous monitoring, automated retraining, model versioning, and observability—can be applied to mobility prediction tasks. By combining traffic simulation, gradient boosting models, online drift detection, and interactive visualization, the system provides a template for deploying adaptive ML in transportation applications.

The time-lapse simulation capability ($300\times$ speed) enables rapid evaluation of drift scenarios that would take days or weeks to observe in real-time, accelerating the development and validation of drift adaptation strategies.

6.2 Limitations

6.2.1 Simulation Fidelity

While SUMO provides realistic traffic dynamics, it simplifies several real-world phenomena:

- **Driver Behavior Heterogeneity:** SUMO’s car-following models use uniform parameters; real drivers exhibit diverse behaviors
- **Network Accuracy:** OSM data may contain inaccuracies in speed limits, lane counts, and traffic signals
- **External Factors:** Weather effects beyond friction (visibility, wind), pedestrian interactions, and parking dynamics not modeled

- **Vehicle Diversity:** Only passenger cars simulated; trucks, buses, motorcycles, and mixed traffic excluded

6.2.2 Drift Scenario Coverage

The platform was evaluated on a single drift type (weather-induced friction reduction). Other important drift scenarios remain unexplored:

- **Gradual Drift:** Slow-onset changes like urbanization, infrastructure improvements, or behavioral trends
- **Recurring Drift:** Seasonal patterns, weekly cycles, or event-driven changes
- **Incident-Based Drift:** Accidents, road closures, or construction requiring rapid adaptation
- **Demand Surges:** Rush hour amplification, special events, or public transit disruptions

6.2.3 Retraining Strategy

The current approach uses fixed windows (1 hour post-drift) and full model retraining. More sophisticated strategies exist:

- **Windowing:** Sliding windows, decay weights, or adaptive sizing based on drift magnitude
- **Incremental Learning:** Online updates without full retraining (not supported by LightGBM)
- **Ensemble Management:** Maintaining multiple models and dynamically weighting predictions
- **Transfer Learning:** Fine-tuning pre-trained models rather than training from scratch

6.2.4 Evaluation Scope

Experiments focused on a single geographic region (central Athens) and prediction task (ETA). Generalization to other cities, traffic patterns, or mobility tasks (fuel consumption, charging station demand, delay propagation) requires further validation.

6.3 Future Work

6.3.1 Short-Term Extensions

Multi-City Evaluation

Expand dataset generation to diverse urban environments (dense European cities, sprawling American cities, grid-based Asian cities) to validate feature engineering and drift detection robustness across different traffic patterns, infrastructure layouts, and demand distributions.

Additional Drift Scenarios

Implement controlled experiments for:

- Gradual drift (slow friction decay over days)

- Recurring drift (rush hour amplification, day/night patterns)
- Spatial drift (localized incidents, construction zones)
- Demand drift (special events, public transit outages)

Advanced Retraining Strategies

Explore alternative adaptation mechanisms:

- Windowing strategies: exponential decay, sliding windows with overlap
- Ensemble methods: maintaining model pools, dynamic weighting
- Online learning: integrate River’s incremental models (HoeffdingTree, AMFRegressor)
- Hybrid approaches: combine batch retraining with online updates

Real-World Validation

Deploy the platform with real taxi or delivery fleet data:

- Compare simulation drift vs. real-world drift characteristics
- Validate detection latency and retraining effectiveness on production data
- Measure operational benefits: prediction accuracy, computational costs, user satisfaction

6.3.2 Medium-Term Research

Multi-Task Learning

Extend the platform to predict multiple related outputs:

- Joint prediction of ETA, fuel consumption, and number of stops
- Shared feature engineering and drift detection across tasks
- Task-specific retraining vs. unified model updates

Explainable Drift

Integrate interpretability methods to diagnose drift causes:

- Feature-level drift: which input distributions changed (e.g., traffic density, route characteristics)
- Concept-level drift: how relationships changed (e.g., speed-flow curves)
- Root cause analysis: linking drift to external events (weather, incidents, policy changes)

Federated Learning for Mobility

Adapt the platform for privacy-preserving multi-fleet learning:

- Local drift detection per fleet, global model aggregation
- Differential privacy for sensitive location data
- Asynchronous updates and Byzantine-robust aggregation

Active Learning for Drift

Optimize data collection during drift:

- Query strategies: select most informative trips for labeling
- Uncertainty-based sampling: prioritize uncertain predictions
- Balance exploration (new regions) vs. exploitation (confident regions)

6.3.3 Long-Term Vision

Self-Healing Mobility Systems

Develop autonomous adaptation systems that:

- Detect, diagnose, and adapt to drift without human intervention
- Generate explanations and confidence estimates for adaptations
- Roll back changes if retraining degrades performance
- Learn meta-policies for when/how to adapt based on drift characteristics

Simulation-in-the-Loop Operations

Integrate SUMO directly into production systems:

- Real-time what-if analysis: simulate alternative routes or policies
- Drift forecasting: predict drift onset based on planned events (construction, weather forecasts)
- Synthetic data augmentation: generate additional training samples on-demand
- Safe exploration: evaluate new models in simulation before production deployment

Universal Mobility Intelligence

Generalize the platform architecture to diverse mobility domains:

- Public transit: delay prediction, ridership forecasting, vehicle assignment
- Micro-mobility: e-scooter demand, battery swapping, rebalancing
- Aviation: flight delay prediction, gate assignment, crew scheduling
- Maritime: vessel ETA, port congestion, emission estimation

Each domain exhibits concept drift (weather, seasonality, policy changes), and the unified platform architecture could enable cross-domain transfer learning and shared infrastructure.

6.4 Closing Remarks

Concept drift is an inherent challenge in deploying machine learning models for real-world mobility systems. This thesis demonstrated that combining simulation-driven evaluation, ensemble drift detection, automated retraining, and interactive visualization can create robust, adaptive prediction platforms.

The key insight is that *simulation is not just for initial model development—it is a powerful tool for evaluating, validating, and improving production ML systems before they face real-world drift*. By manufacturing controlled drift scenarios, we can systematically test adaptation strategies, quantify detection latency, and measure retraining effectiveness in ways that would be impossible or prohibitively expensive with live data.

As intelligent transportation systems become increasingly data-driven, the ability to detect and adapt to distributional shifts will be critical for maintaining service quality, safety, and user trust. This thesis provides a foundation for building such adaptive systems and opens multiple avenues for future research in simulation-driven MLOps, explainable drift detection, and self-healing mobility intelligence.

The code, datasets, and experimental configurations developed for this thesis are available as open-source resources to support reproducibility and enable the research community to build upon these contributions.

Bibliography

- [1] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. “A survey on concept drift adaptation”. In: *ACM Computing Surveys (CSUR)* 46.4 (2014), pp. 1–37.
- [2] Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., and Zhang, G. “Learning under concept drift: A review”. In: *IEEE Transactions on Knowledge and Data Engineering* 31.12 (2018), pp. 2346–2363.
- [3] Webb, G. I., Hyde, R., Cao, H., Nguyen, H., and Petitjean, F. “Characterizing concept drift”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 1285–1294.
- [4] Ross, G. J. “Exponential smoothing for concept drift adaptation”. In: *Expert Systems with Applications* 40.6 (2013), pp. 2006–2016.
- [5] Montiel, J., Read, J., Bifet, A., and Abdesslem, T. “River: machine learning for streaming data in Python”. In: *Journal of Machine Learning Research* 22.1 (2021), pp. 1–7.
- [6] Li, Y. and Rose, G. “Short-term travel time prediction on freeway segments based on adaptive Kalman filter”. In: *Transportation Research Record* 2165.1 (2012), pp. 69–78.
- [7] Angelis, G. *Fuel Consumption Prediction Model*. Unpublished work, integrated in drift-aware platform. 2025.
- [8] Tzelepis, S. *Number of Stops Prediction Model*. Unpublished work, integrated in drift-aware platform. 2025.
- [9] Krajzewicz, D., Erdmann, J., Behrisch, M., and Bieker, L. “Recent development and applications of SUMO - Simulation of Urban MObility”. In: *International Journal On Advances in Systems and Measurements* 5.3&4 (2012), pp. 128–138.
- [10] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. 2017, pp. 3146–3154.